

Complete CSS Notes

Types of CSS, Typography, Cascade, Box Model, Display and Positioning

Student Learning Material

Detailed theory, syntax, examples, comparisons, common mistakes, practice tasks and interview questions.

Topics Covered

1. Types of CSS
2. CSS Typography
3. CSS Cascade
4. CSS Box Model
5. CSS Display
6. CSS Positioning

1. Introduction to CSS

CSS stands for Cascading Style Sheets. It is used to control the appearance and layout of HTML elements on a webpage.

CSS can be used to change:

- Text color and font
- Background color and image
- Width and height
- Margin and padding
- Borders and shadows
- Element alignment
- Page layout
- Element positioning
- Responsive behavior
- Transitions and animations

Basic CSS Syntax

```
selector {  
  property: value;  
}
```

Example:

```
h1 {  
  color: blue;  
  font-size: 36px;  
}
```

In this example:

- **h1** is the selector.
- **color** and **font-size** are properties.
- **blue** and **36px** are values.
- Each **property: value** pair is called a declaration.
- All declarations inside **{ }** form a declaration block.

2. Types of CSS

CSS can be applied to HTML in three main ways:

1. Inline CSS
2. Internal CSS
3. External CSS

2.1 Inline CSS

Inline CSS is written directly inside an HTML element using the `style` attribute.

Syntax

```
<tag style="property: value;">Content</tag>
```

Example

```
<h1 style="color: blue; text-align: center;">
  Welcome to CSS
</h1>

<p style="color: red; font-size: 18px;">
  This paragraph uses inline CSS.
</p>
```

Advantages

- Quick for small changes.
- Useful for testing a style.
- Affects a specific element directly.
- Does not require a separate stylesheet.

Disadvantages

- Makes HTML lengthy and difficult to read.
- Styles must be repeated for multiple elements.
- Difficult to maintain in a large project.
- Mixes HTML structure with presentation.
- Does not provide good reusability.

When to Use

Use inline CSS for quick testing or a small, element-specific change. Avoid using it to style an entire website.

2.2 Internal CSS

Internal CSS is written inside a `<style>` element, normally in the `<head>` section of the HTML document.

Example

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Internal CSS</title>

  <style>
    body {
      background-color: #f4f4f4;
      font-family: Arial, sans-serif;
    }

    h1 {
      color: navy;
      text-align: center;
    }

    p {
      color: darkgreen;
      font-size: 18px;
    }
  </style>
</head>
<body>
  <h1>Internal CSS Example</h1>
  <p>This paragraph uses internal CSS.</p>
</body>
</html>
```

Advantages

- Keeps the styles for one page in one place.
- Cleaner than applying inline CSS repeatedly.
- Supports selectors, classes, IDs, pseudo-classes, and media queries.
- Useful for single-page projects and demonstrations.

Disadvantages

- Styles are limited to one HTML document.
- The same styles must be repeated in other pages.
- Makes the HTML file larger.
- Difficult to maintain across a multi-page website.

When to Use

Use internal CSS for a single webpage, a small project, a demonstration, or page-specific styling.

2.3 External CSS

External CSS is written in a separate file with the `.css` extension. The stylesheet is connected to the HTML document using the `<link>` element.

Project Structure

```
css-project/  
├─ index.html  
└─ style.css
```

style.css

```
body {  
    background-color: #f5f5f5;  
    font-family: Arial, sans-serif;  
}  
  
h1 {  
    color: purple;  
    text-align: center;  
}  
  
p {  
    color: #333333;  
    font-size: 18px;  
}
```

index.html

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
    <meta charset="UTF-8">  
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  
    <title>External CSS</title>  
  
    <link rel="stylesheet" href="style.css">  
</head>  
<body>  
    <h1>External CSS Example</h1>  
    <p>This paragraph uses external CSS.</p>  
</body>  
</html>
```

Understanding the Link Element

```
<link rel="stylesheet" href="style.css">
```

- **link** connects an external resource to the HTML document.
- **rel="stylesheet"** identifies the resource as a stylesheet.
- **href="style.css"** contains the path to the CSS file.

Advantages

- One stylesheet can be used by multiple pages.
- Keeps HTML clean and readable.
- Provides reusable styles.
- Reduces duplicate CSS.
- Makes website-wide changes easier.
- Suitable for large and professional projects.

Disadvantages

- The page may appear unstyled if the file path is incorrect.

- An extra file must be created and managed.
- The stylesheet must be loaded by the browser.

Comparison

Feature	Inline CSS	Internal CSS	External CSS
Location	HTML element	<code><style></code> element	Separate <code>.css</code> file
Scope	One element	One page	Multiple pages
Reusable	No	Limited	Yes
Maintenance	Difficult	Moderate	Easy
Suitable for large projects	No	No	Yes

3. CSS Typography

Typography is the process of styling and arranging text to make it readable, clear, and visually attractive.

CSS typography includes:

- Font family
- Font size
- Font weight
- Font style
- Line height
- Letter spacing
- Word spacing
- Text alignment
- Text decoration
- Text transformation
- Text indentation
- Text shadow
- Text overflow

3.1 Font Family

The `font-family` property defines the typeface used for text.

```
body {
  font-family: Arial, Helvetica, sans-serif;
}
```

Multiple fonts are provided as fallbacks. If the browser cannot use `Arial`, it tries `Helvetica`, followed by a generic `sans-serif` font.

A font name containing spaces should be placed in quotes:

```
h1 {  
  font-family: "Times New Roman", serif;  
}
```

Generic Font Families

- `serif`
- `sans-serif`
- `monospace`
- `cursive`
- `fantasy`
- `system-ui`

3.2 Font Size

The `font-size` property controls the size of text.

```
h1 {  
  font-size: 40px;  
}  
  
p {  
  font-size: 1rem;  
}
```

Common CSS Units

- `px`: fixed pixel value
- `%`: percentage relative to a parent value
- `em`: relative to the computed font size of the parent
- `rem`: relative to the root element font size
- `vw`: relative to viewport width
- `vh`: relative to viewport height

Example:

```
html {  
  font-size: 16px;  
}  
  
h1 {  
  font-size: 2rem;  
}
```

Here, `2rem` is calculated from the font size of the root `<html>` element.

3.3 Font Weight

The `font-weight` property controls the thickness of text.

```
p {  
  font-weight: normal;  
}
```

```
strong {
  font-weight: bold;
}

h1 {
  font-weight: 700;
}
```

Common values include:

```
normal
bold
100 to 900
```

The available numeric weights depend on the selected font.

3.4 Font Style

The **font-style** property is commonly used to make text italic.

```
.normal-text {
  font-style: normal;
}

.italic-text {
  font-style: italic;
}

.oblique-text {
  font-style: oblique;
}
```

3.5 Line Height

The **line-height** property controls the vertical space between lines of text.

```
p {
  font-size: 18px;
  line-height: 1.6;
}
```

A unitless value such as **1.6** is commonly used because it scales with the element's font size.

3.6 Letter and Word Spacing

```
h1 {
  letter-spacing: 2px;
}

p {
  word-spacing: 6px;
}
```

- **letter-spacing** controls space between characters.
- **word-spacing** controls space between words.

3.7 Text Alignment

```
.left {
  text-align: left;
}

.center {
  text-align: center;
}

.right {
  text-align: right;
}

.justified {
  text-align: justify;
}
```

text-align aligns inline content inside a block-level container. It does not position the block element itself.

3.8 Text Decoration

```
a {
  text-decoration: none;
}

.underlined {
  text-decoration: underline;
}

.deleted {
  text-decoration: line-through;
}
```

Individual decoration properties can also be used:

```
.title {
  text-decoration-line: underline;
  text-decoration-color: red;
  text-decoration-style: wavy;
  text-decoration-thickness: 2px;
}
```

3.9 Text Transformation

```
.uppercase {
  text-transform: uppercase;
}

.lowercase {
  text-transform: lowercase;
}

.capitalize {
  text-transform: capitalize;
}
```

This changes the displayed capitalization without changing the original HTML text.

3.10 Text Indentation

```
p {  
    text-indent: 40px;  
}
```

This adds indentation to the first line of a text block.

3.11 Text Shadow

```
h1 {  
    text-shadow: 2px 2px 4px gray;  
}
```

Syntax:

text-shadow: horizontal-offset vertical-offset blur-radius color;

Multiple shadows can be added with commas:

```
h1 {  
    text-shadow: 1px 1px 2px black, 0 0 8px blue;  
}
```

3.12 Text Overflow

Text overflow is useful when text does not fit inside a container.

```
.title {  
    width: 250px;  
    white-space: nowrap;  
    overflow: hidden;  
    text-overflow: ellipsis;  
}
```

Long text is displayed with an ellipsis when all required properties are applied.

Typography Example

```
<article class="article">  
    <h1 class="article-title">Introduction to CSS Typography</h1>  
    <p class="article-text">  
        Typography improves the readability and appearance of webpage content.  
    </p>  
    <a class="article-link" href="#">Read more</a>  
</article>
```

```
.article {  
    font-family: Arial, sans-serif;  
}
```

```
.article-title {  
    font-size: 2rem;  
    font-weight: 700;
```

```
    line-height: 1.2;
    letter-spacing: 1px;
    text-align: center;
}

.article-text {
    font-size: 1rem;
    line-height: 1.7;
    color: #333333;
}

.article-link {
    font-weight: 600;
    text-decoration: none;
    text-transform: uppercase;
}
```

4. CSS Cascade

The cascade is the process used by the browser to decide which CSS declaration should be applied when multiple rules target the same element.

The word "cascading" indicates that styles can come from different sources and compete with one another.

The result is influenced by:

1. Origin and importance
2. Specificity
3. Source order
4. Inheritance

4.1 Source Order

When two declarations have the same importance and specificity, the declaration written later normally wins.

```
p {
    color: blue;
}

p {
    color: red;
}
```

The paragraph becomes red because the second rule appears later.

4.2 Specificity

Specificity measures how precisely a selector targets an element.

A simplified order is:

Inline style > ID selector > Class, attribute, and pseudo-class selectors > Element and pseudo-element selectors

Example HTML:

```
<p id="message" class="note">Learning CSS</p>
```

Example CSS:

```
p {  
    color: green;  
}  
  
.note {  
    color: blue;  
}  
  
#message {  
    color: red;  
}
```

The text becomes red because **#message** is more specific.

Specificity Examples

```
p { }  
.note { }  
#message { }  
main .card p { }
```

Avoid creating unnecessarily complicated selectors because they are difficult to override and maintain.

4.3 Inline Styles

Inline styles usually override normal declarations written in internal or external stylesheets.

```
<p class="message" style="color: red;">Hello</p>
```

```
.message {  
    color: blue;  
}
```

The text is red because the inline declaration has greater specificity than the normal class declaration.

4.4 The **!important** Keyword

```
p {  
    color: blue !important;  
}
```

!important increases the importance of a declaration. It should be used carefully because excessive use makes styles harder to override and debug.

```
.message {  
    color: blue !important;  
}  
  
#message {
```

```
    color: red;
}
```

The class declaration can win in this example because it is marked **!important** while the ID declaration is not.

If competing declarations are both important, specificity and source order are considered again.

4.5 Inheritance

Some CSS properties are inherited from a parent element by default.

```
<div class="container">
  <p>This paragraph can inherit the text color.</p>
</div>
```

```
.container {
  color: purple;
  font-family: Arial, sans-serif;
}
```

Text-related properties such as **color** and **font-family** are commonly inherited. Layout properties such as **margin**, **padding**, **border**, **width**, and **height** are generally not inherited automatically.

Inheritance Keywords

```
.child {
  color: inherit;
}
```

Useful global values include:

- **inherit**: uses the computed value from the parent
- **initial**: uses the property's initial value
- **unset**: behaves like **inherit** for inherited properties and **initial** for others
- **revert**: rolls the property back to the value from an earlier cascade origin or layer

4.6 Internal CSS vs External CSS

Internal CSS does not automatically have a higher priority than external CSS.

If the selectors have equal importance and specificity, source order decides the result.

```
<link rel="stylesheet" href="style.css">

<style>
  p {
    color: blue;
  }
</style>
```

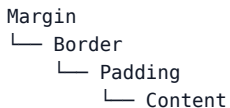
If **style.css** contains **p { color: green; }**, the internal rule appears later and normally wins.

5. CSS Box Model

Every visible HTML element can be treated as a rectangular box. The CSS box model explains how the size and spacing of that box are calculated.

The box model contains four areas:

1. Content
2. Padding
3. Border
4. Margin



5.1 Content

The content area contains text, images, or child elements.

```
.box {  
  width: 300px;  
  height: 150px;  
}
```

By default, **width** and **height** apply to the content area when **box-sizing** is **content-box**.

5.2 Padding

Padding is the space between content and border.

```
.box {  
  padding: 20px;  
}
```

Individual sides:

```
.box {  
  padding-top: 10px;  
  padding-right: 20px;  
  padding-bottom: 10px;  
  padding-left: 20px;  
}
```

Padding Shorthand

```
/* All sides */  
padding: 20px;  
  
/* Vertical | Horizontal */  
padding: 10px 20px;  
  
/* Top | Horizontal | Bottom */  
padding: 10px 20px 30px;  
  
/* Top | Right | Bottom | Left */
```

```
padding: 10px 20px 30px 40px;
```

The element's background is painted through the padding area unless other background settings change that behavior.

5.3 Border

The border surrounds the content and padding.

```
.box {  
  border: 2px solid black;  
}
```

The shorthand contains:

```
border-width border-style border-color
```

Common border styles:

- **solid**
- **dashed**
- **dotted**
- **double**
- **none**

Rounded corners:

```
.box {  
  border-radius: 12px;  
}
```

5.4 Margin

Margin is the transparent space outside the border. It creates separation between elements.

```
.box {  
  margin: 20px;  
}
```

Margin shorthand follows the same one-to-four value pattern as padding.

Horizontal Centering

A block element with a defined width can often be centered horizontally using:

```
.box {  
  width: 400px;  
  margin: 0 auto;  
}
```

Margin Collapse

Vertical margins of certain block elements can collapse, meaning the resulting gap may be based on one margin rather than the sum of both margins. Padding, flex layout, grid layout, borders, or

other formatting conditions can change this behavior.

5.5 Total Box Size

With the default **content-box** model:

```
.box {  
  width: 300px;  
  padding: 20px;  
  border: 5px solid black;  
  margin: 10px;  
}
```

Rendered width excluding margins:

$300 + 20 + 20 + 5 + 5 = 350\text{px}$

Total horizontal space including margins:

$350 + 10 + 10 = 370\text{px}$

5.6 Box Sizing

Content Box

```
.box {  
  box-sizing: content-box;  
}
```

The declared width and height apply only to the content area. Padding and borders are added outside them.

Border Box

```
.box {  
  box-sizing: border-box;  
}
```

The declared width and height include content, padding, and border. This often makes layout calculations easier.

A common project reset is:

```
*,  
*::before,  
*::after {  
  box-sizing: border-box;  
}
```

Box Model Example

```
<div class="card">  
  <h2>Student Profile</h2>  
  <p>Frontend Development Student</p>  
</div>
```

```
</div>
```

```
.card {  
  box-sizing: border-box;  
  width: 320px;  
  padding: 24px;  
  border: 2px solid #333333;  
  margin: 30px auto;  
  border-radius: 10px;  
  background-color: #f5f5f5;  
}
```

6. CSS Display

The **display** property determines how an element participates in page layout.

Common values include:

- **block**
- **inline**
- **inline-block**
- **none**
- **flex**
- **grid**

6.1 Block

A block-level element normally:

- Starts on a new line.
- Uses the available horizontal space by default.
- Accepts width and height.
- Accepts margin and padding on all sides.

Common block elements include:

```
<div>, <p>, <h1> to <h6>, <section>, <article>, <header>, <footer>
```

```
.box {  
  display: block;  
  width: 300px;  
  height: 100px;  
}
```

6.2 Inline

An inline element normally:

- Remains on the same line as nearby inline content.
- Uses only the space needed by its content.
- Does not behave like a block with respect to width and height.
- Has special behavior for vertical margin and padding within line layout.

Common inline elements include:

```
<span>, <a>, <strong>, <em>, <label>
```

```
.highlight {  
  display: inline;  
  color: red;  
}
```

6.3 Inline Block

inline-block combines important characteristics of inline and block layout.

An inline-block element:

- Can remain beside other inline or inline-block elements.
- Accepts width and height.
- Accepts margin and padding on all sides.

```
.button {  
  display: inline-block;  
  width: 140px;  
  padding: 12px;  
  text-align: center;  
}
```

6.4 Display None

```
.hidden {  
  display: none;  
}
```

The element is not displayed and does not occupy layout space.

Compare it with:

```
.invisible {  
  visibility: hidden;  
}
```

With **visibility: hidden**, the element is invisible but its layout space is generally preserved.

6.5 Flex

Flexbox is a one-dimensional layout system used to arrange items in a row or column.

```
.container {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  gap: 20px;  
}
```

Common flex container properties:

- `flex-direction`
- `justify-content`
- `align-items`
- `align-content`
- `flex-wrap`
- `gap`

Example:

```
.navigation {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 16px;
}
```

6.6 Grid

CSS Grid is a two-dimensional layout system used to arrange content in rows and columns.

```
.container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
}
```

Common grid properties:

- `grid-template-columns`
- `grid-template-rows`
- `gap`
- `grid-column`
- `grid-row`
- `justify-items`
- `align-items`

Responsive example:

```
.card-container {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 20px;
}
```

Display Comparison

Display Value	New Line	Width and Height	Basic Use
<code>block</code>	Yes	Supported	Sections and containers
<code>inline</code>	No	Limited in inline flow	Text-level elements
<code>inline-block</code>	No	Supported	Buttons and small boxes

Display Value	New Line	Width and Height	Basic Use
<code>none</code>	Not displayed	Not applicable	Remove from layout
<code>flex</code>	Block-level by default	Supported	One-dimensional layout
<code>grid</code>	Block-level by default	Supported	Two-dimensional layout

7. CSS Positioning

The `position` property controls how an element is positioned in a document.

Main values:

1. `static`
2. `relative`
3. `absolute`
4. `fixed`
5. `sticky`

Offset properties include:

```
top: 0;
right: 0;
bottom: 0;
left: 0;
```

These offsets behave according to the selected positioning method.

7.1 Static Positioning

`static` is the default position value.

```
.box {
  position: static;
}
```

A statically positioned element follows the normal document flow. The `top`, `right`, `bottom`, and `left` offsets do not reposition a normal static element.

7.2 Relative Positioning

A relatively positioned element remains in normal document flow but can be visually offset from its original position.

```
.box {
  position: relative;
  top: 10px;
  left: 20px;
}
```

Its original layout space remains reserved.

A relative parent is also commonly used as the containing block for an absolutely positioned child.

7.3 Absolute Positioning

An absolutely positioned element is removed from normal document flow.

```
.parent {
  position: relative;
}

.child {
  position: absolute;
  top: 10px;
  right: 10px;
}
```

The child is positioned using its containing block. A positioned ancestor, such as a parent with **position: relative**, is commonly used to control that reference area.

Example: Badge

```
<div class="product-card">
  <span class="badge">New</span>
  <h2>CSS Course</h2>
</div>
```

```
.product-card {
  position: relative;
  width: 300px;
  padding: 30px;
  border: 1px solid #cccccc;
}

.badge {
  position: absolute;
  top: 10px;
  right: 10px;
  padding: 4px 8px;
  background-color: red;
  color: white;
}
```

7.4 Fixed Positioning

A fixed element is removed from normal flow and positioned relative to the viewport.

```
.help-button {
  position: fixed;
  right: 20px;
  bottom: 20px;
}
```

It generally stays in the same viewport location while the page scrolls.

Common uses:

- Floating action buttons

- Persistent navigation
- Back-to-top buttons
- Help or chat controls

7.5 Sticky Positioning

Sticky positioning behaves like relative positioning until a scrolling threshold is reached. It then sticks within its scrolling context.

```
header {
  position: sticky;
  top: 0;
  background-color: white;
  z-index: 100;
}
```

A threshold such as `top: 0` is normally needed. Sticky behavior can be affected by the dimensions and overflow settings of ancestor elements.

7.6 Z-Index

The `z-index` property helps control the stacking order of overlapping positioned elements.

```
.header {
  position: fixed;
  top: 0;
  z-index: 1000;
}
```

A higher `z-index` can place an element above another element within the relevant stacking context. Positioning and stacking contexts affect how values are compared.

```
.box-one {
  position: absolute;
  z-index: 1;
}

.box-two {
  position: absolute;
  z-index: 2;
}
```

Within the same stacking context, `.box-two` is placed above `.box-one`.

Position Comparison

Position	In Normal Flow	Reference Area	Scroll Behavior
<code>static</code>	Yes	Normal document flow	Scrolls normally
<code>relative</code>	Yes	Original position	Scrolls normally
<code>absolute</code>	No	Containing block	Scrolls with its context

Position	In Normal Flow	Reference Area	Scroll Behavior
fixed	No	Viewport	Usually remains fixed
sticky	Yes until threshold behavior	Scrolling context	Sticks after threshold

8. Combined Practical Example

Project Structure

```
css-complete-example/
├── index.html
└── style.css
```

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>CSS Complete Example</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="site-header">
    <h1>CSS Learning Notes</h1>
  </header>

  <main class="card-container">
    <article class="card">
      <span class="badge">New</span>
      <h2>Typography</h2>
      <p>Learn how to style and arrange text using CSS.</p>
      <a class="button" href="#">Read Notes</a>
    </article>

    <article class="card">
      <h2>Box Model</h2>
      <p>Understand content, padding, border, and margin.</p>
      <a class="button" href="#">Read Notes</a>
    </article>
  </main>

  <button class="help-button">Help</button>
</body>
</html>
```

style.css

```
*,
*::before,
*::after {
  box-sizing: border-box;
}
```

```

body {
  margin: 0;
  background-color: #f4f6f8;
  color: #222222;
  font-family: Arial, sans-serif;
  line-height: 1.6;
}

.site-header {
  position: sticky;
  top: 0;
  z-index: 100;
  padding: 16px;
  background-color: #1f3c88;
  color: white;
  text-align: center;
}

.site-header h1 {
  margin: 0;
  font-size: 2rem;
  letter-spacing: 1px;
}

.card-container {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 24px;
  max-width: 1000px;
  margin: 40px auto;
  padding: 20px;
}

.card {
  position: relative;
  padding: 24px;
  border: 1px solid #dddddd;
  border-radius: 12px;
  background-color: white;
}

.card h2 {
  margin-top: 0;
  font-size: 1.5rem;
}

.badge {
  position: absolute;
  top: 12px;
  right: 12px;
  padding: 4px 8px;
  border-radius: 4px;
  background-color: #d62828;
  color: white;
  font-size: 12px;
  font-weight: bold;
  text-transform: uppercase;
}

.button {
  display: inline-block;
  padding: 10px 18px;
  border-radius: 6px;
  background-color: #1f3c88;
  color: white;
  font-weight: bold;
  text-decoration: none;
}

```

```
.help-button {
  position: fixed;
  right: 20px;
  bottom: 20px;
  padding: 12px 20px;
  border: none;
  border-radius: 24px;
  background-color: #198754;
  color: white;
  cursor: pointer;
}
```

9. Common Mistakes

Incorrect CSS File Path

```
<link rel="stylesheet" href="style.css">
```

Confirm that the CSS file is actually in the same folder as the HTML file. If it is inside a **css** folder, use:

```
<link rel="stylesheet" href="css/style.css">
```

Forgetting Units

Incorrect:

```
p {
  font-size: 18;
}
```

Correct:

```
p {
  font-size: 18px;
}
```

Some properties accept unitless values, but a length value generally needs a valid unit.

Confusing Margin and Padding

- Margin creates space outside the border.
- Padding creates space inside the border.

Expecting Width to Work Normally on Inline Elements

If width and height are needed while keeping elements on the same line, consider **inline-block**, **flexbox**, or **grid**.

Absolute Positioning Without a Controlled Parent

Add positioning to the parent when the child should be positioned inside it:

```
.parent {  
  position: relative;  
}  
  
.child {  
  position: absolute;  
  top: 0;  
  right: 0;  
}
```

Using Z-Index Without Understanding Stacking Context

A large **z-index** does not always escape the stacking context created by an ancestor. Inspect the positioned ancestors when layering does not behave as expected.

Overusing **!important**

Prefer readable selectors and proper source order. Use **!important** only when there is a clear reason.

10. Best Practices

- Use external CSS for multi-page projects.
- Apply a consistent **box-sizing** rule.
- Use meaningful class names.
- Keep selector specificity manageable.
- Use relative units where appropriate.
- Set a readable line height for paragraphs.
- Use flexbox for one-dimensional layouts.
- Use grid for row-and-column layouts.
- Use positioning only when normal flow, flexbox, or grid does not solve the requirement.
- Organize CSS into logical sections.
- Add comments for complex rules.
- Test layouts at different viewport sizes.

Example comment:

```
/* =====  
  Student card component  
  ===== */
```

11. Practice Tasks

Task 1: Typography Card

Create a card containing a heading, paragraph, and link. Apply:

- `font-family`
- `font-size`
- `font-weight`
- `line-height`
- `letter-spacing`
- `text-decoration`
- `text-transform`

Task 2: Box Model

Create three boxes and apply different values for:

- Width and height
- Padding
- Border
- Margin
- `border-radius`
- `box-sizing`

Compare `content-box` with `border-box`.

Task 3: Display Property

Create navigation links and display them using:

- `inline`
- `inline-block`
- `flex`

Observe the difference in spacing and dimension control.

Task 4: Positioning

Create a product card with:

- A relatively positioned card
- An absolutely positioned discount badge
- A sticky page header
- A fixed help button

Task 5: Complete Page

Create a responsive course listing page with:

- External CSS
- Styled headings and paragraphs
- Card box models

- A grid card container
- An absolute card badge
- A sticky header
- A fixed action button

12. Interview Questions and Answers

What are the three types of CSS?

Inline CSS, internal CSS, and external CSS.

Which type of CSS is recommended for a multi-page website?

External CSS because it provides reusable styles and easier maintenance.

What is typography in CSS?

Typography is the styling and arrangement of text using properties such as font family, font size, line height, spacing, alignment, and decoration.

What is the CSS cascade?

The cascade is the process the browser uses to choose the winning declaration when multiple CSS rules apply to the same element.

What is specificity?

Specificity is the weight calculated from a selector that helps determine which competing declaration is applied.

Does internal CSS always override external CSS?

No. Importance, specificity, and source order determine the result.

What are the four parts of the box model?

Content, padding, border, and margin.

What is the difference between margin and padding?

Margin is outside the border, while padding is between the content and the border.

What is the difference between **content-box** and **border-box**?

With **content-box**, declared dimensions apply to content and padding and border are added.
With **border-box**, declared dimensions include content, padding, and border.

What is the difference between **block** and **inline**?

A block element normally starts on a new line and can accept dimensions. An inline element stays within text flow and uses only the space required by its content.

What is the difference between **display: none** and **visibility: hidden**?

display: none removes the element from layout. **visibility: hidden** hides it while generally preserving its layout space.

What is the difference between **relative** and **absolute** positioning?

A relative element remains in normal flow and is offset from its original position. An absolute element is removed from normal flow and positioned using its containing block.

What is **fixed** positioning?

Fixed positioning removes an element from normal flow and positions it relative to the viewport.

What is **sticky** positioning?

Sticky positioning behaves like relative positioning until a scrolling threshold is reached, after which it sticks within its scrolling context.

What is **z-index**?

z-index helps define the stacking order of overlapping elements within applicable stacking contexts.

13. Quick Revision

Types of CSS:

Inline -> style attribute

Internal -> <style> element

External -> separate .css file

Typography:

font-family, font-size, font-weight, font-style,

line-height, letter-spacing, word-spacing,
text-align, text-decoration, text-transform

Cascade:

Origin and importance -> Specificity -> Source order
Inheritance may supply values from a parent

Box Model:

Content -> Padding -> Border -> Margin

Display:

block, inline, inline-block, none, flex, grid

Position:

static, relative, absolute, fixed, sticky

Conclusion

These CSS concepts form the foundation of webpage styling and layout:

- Types of CSS define where styles are written.
- Typography controls the appearance and readability of text.
- The cascade resolves competing style declarations.
- The box model determines element dimensions and spacing.
- Display controls an element's layout behavior.
- Positioning controls how elements are placed and layered.

Understanding these topics makes it easier to create clean, responsive, and maintainable user interfaces.